

ENTERPRISE FRONTEND ENGINEERING HANDBOOK

Building scalable, secure, performant and maintainable Next.js applications

Portfolio Edition. A vendor-neutral architecture and engineering reference designed to demonstrate senior-level frontend thinking, practical standards, and implementation discipline.

Ashish Jadhav

Frontend Engineer | React | Next.js | TypeScript | Architecture

About This Handbook

This handbook presents a practical framework for building and modernising large-scale frontend applications. It combines architecture, security, design systems, API design, state management, performance, testing, accessibility, CI/CD, observability, and engineering governance into one coherent operating model.

Public-safe by design. The material is deliberately generic and does not include proprietary company names, internal code, credentials, customer information, or organisation-specific implementation details.

How to use it

- Use it as a portfolio artefact demonstrating architectural depth.
- Publish the Markdown chapters on GitHub for easy browsing.
- Provide the PDF as a downloadable reference.
- Adapt the included portfolio page into your personal Next.js site.
- Extend individual chapters with public demo code over time.

Contents

01. Engineering Principles and Decision-Making

02. Next.js Application Architecture

03. Rendering and Data Delivery Strategy

04. Design System and Global CSS

05. Component Architecture

06. TypeScript Engineering Standards

07. API and Repository Architecture

08. State Management

09. Authentication and Authorization

10. Frontend Security

11. Performance Engineering

12. Testing Strategy

13. Accessibility

14. Forms and Validation

- 15. Data Tables and Search Experiences**
- 16. Observability and Operational Reliability**
- 17. CI/CD and Release Engineering**
- 18. Developer Experience and Repository Governance**
- 19. Code Review Standard**
- 20. Legacy Modernization**
- 21. Reference Implementation Playbook**
- 22. Portfolio Case Study Guidance**

1. Engineering Principles and Decision-Making

The foundational rules that keep a frontend platform coherent as products, traffic, and teams grow.

What enterprise-grade means

Enterprise quality is not defined by folder depth or framework popularity. It is the combination of trusted security boundaries, explicit ownership, predictable delivery, measurable performance, automated regression protection, and accessible user experiences.

The architecture should make the safest and most maintainable path the easiest path for engineers.

Core principles

- Prefer incremental modernization over disruptive rewrites.
- Treat browser code as untrusted.
- Use one approved pattern per cross-cutting concern.
- Keep business rules testable outside React.
- Make quality gates executable in CI.
- Measure architecture through outcomes, not diagrams alone.

Architecture decision records

Record context, options, decision, consequences, migration plan, owner, and review date for major choices.

Use ADRs for state libraries, rendering strategy, API clients, design system foundations, authentication, and testing tools.

2. Next.js Application Architecture

A feature-oriented, server-first architecture for modern Next.js applications.

Target structure

Route files compose features and enforce route boundaries. Feature modules own business capabilities. Shared modules contain intentionally generic primitives only.

Recommended folder tree

```
src/  
  app/  
    (public)/  
    (authenticated)/  
    api/  
    layout.tsx  
  features/  
    accounts/  
    reporting/  
    workflows/  
  shared/  
    api/ auth/ config/ ui/ styles/ types/  
  store/  
  tests/  
  middleware.ts
```

Boundary rules

- Features may import shared modules; shared modules must not import features.
- Cross-feature usage must go through a public index.ts.
- Components do not call arbitrary endpoints directly.
- Transport types are mapped to domain models.
- Legacy code is isolated behind adapters with removal targets.

Server and client components

Default to Server Components for read-heavy routes and layout composition. Use Client Components only where browser APIs, event handlers, local interaction state, or client libraries require them.

Keep client boundaries narrow to reduce hydration cost and bundle size.

3. Rendering and Data Delivery Strategy

Choose SSR, static generation, streaming, and client rendering intentionally.

Rendering decision matrix

Scenario	Preferred strategy	Reason
Public marketing page	Static generation	Fast, cacheable, stable content
Authenticated dashboard	Server rendering + client islands	Fresh data with limited hydration
Large interactive builder	Client component loaded dynamically	Heavy interaction and browser-only libraries
Frequently updated report	Server rendering with cache policy	Freshness and predictable request cost
Long-running data view	Streaming + Suspense	Progressive display and lower perceived latency

Caching rules

- Define freshness by data class rather than adding cache settings ad hoc.
- Use request memoization, route caching, CDN caching, and client caching only when ownership and invalidation are explicit.
- Mutations must invalidate or update every affected query deterministically.

4. Design System and Global CSS

Create a single visual language shared by custom components and third-party UI libraries.

Style layers

- globals.css for reset, base typography, document defaults, focus style, and truly global utilities.
- tokens.css for semantic colours, spacing, type, radius, shadow, motion, z-index, and dimensions.
- themes.css for light and dark overrides.
- Feature-level CSS Modules for local layout only.

Token example

```
@layer reset, tokens, base, components, utilities;

@layer tokens {
  :root {
    --font-sans: var(--font-inter), Arial, sans-serif;
    --color-primary: #1646d8;
    --text-primary: #0f1d2e;
    --surface-page: #f6f8fb;
    --surface-card: #ffffff;
    --border-default: #d9e0e8;
    --space-1: .25rem;
    --space-2: .5rem;
    --space-4: 1rem;
    --radius-md: .5rem;
    --shadow-sm: 0 1px 2px rgb(15 29 46 / 8%);
  }
}
```

Governance

- No new hardcoded brand colours, spacing, shadows, or z-index values without a token proposal.
- Use semantic names such as status-error and text-secondary.
- Bridge tokens into MUI, Tailwind, or other component foundations.
- Document every shared component state in Storybook.

5. Component Architecture

Build components that remain understandable, composable, and testable.

Layer model

Layer	Responsibility	Example
Route shell	Routing, metadata, authorization, initial orchestration	page.tsx
Feature container	Coordinates use case and state	ReportContainer
Presentational component	Typed rendering and events	ReportTable
Hook	Reusable UI orchestration	useReportFilters
Domain function	Pure business transformation	buildReportQuery
Repository	Endpoint access and mapping	reportRepository

Component standards

- Prefer explicit props over broad context.
- Avoid components that fetch, transform, validate, navigate, and render simultaneously.
- Use controlled primitives for forms and reusable compound components only when they improve ergonomics.
- Define loading, empty, success, error, disabled, permission, and offline states.

6. TypeScript Engineering Standards

Use TypeScript as a design tool rather than a syntax layer.

Compiler policy

- Enable strict, strictNullChecks, noUncheckedIndexedAccess, exactOptionalPropertyTypes, and noImplicitOverride progressively.
- Create a no-growth baseline for any, unsafe casts, @ts-ignore, and eslint-disable.

Boundary validation

Treat external input as unknown. Validate API responses, search params, local persistence, file content, and postMessage events at runtime.

Map transport contracts into domain types before exposing data to complex UI.

Naming and exports

- PascalCase for components and types, camelCase for functions and values, predictable handler prefixes, and one agreed folder naming convention.
- Prefer named exports for shared modules and public barrel files only at feature boundaries.

7. API and Repository Architecture

Standardise data access, error behaviour, cancellation, retries, and contracts.

Central client requirements

- Base URL resolution, credentials, timeout, request ID, cancellation, normalized errors, and observability hooks.
- Safe GET requests may retry transient failures; mutations must never retry blindly.
- High-impact create or send actions should use idempotency keys when supported.

Normalized error contract

```
type AppError = {  
  code: string;  
  message: string;  
  status?: number;  
  fieldErrors?: Record<string, string>;  
  correlationId?: string;  
  retryable: boolean;  
};
```

Repository example

```
export async function getReport(input: ReportQuery): Promise<Report> {  
  const response = await httpClient.get('/reports', { params: input });  
  const parsed = ReportResponseSchema.parse(response.data);  
  return mapReportResponse(parsed);  
}
```

8. State Management

Assign every state value to the smallest correct owner.

Ownership model

State type	Preferred owner	Examples
Server state	TanStack Query or server-data pattern	Lists, reports, profiles
URL state	Search params	Filters, sort, pagination, tabs
Global workflow state	Redux Toolkit	Multi-route builder draft
Local state	Component or reducer	Dialog, selection, transient UI
Form state	One approved form library	Create/edit workflows
Session state	Server-backed provider	User, account, permissions

Rules

- Do not duplicate server state in Redux.
- Do not use global state for values that can live in a URL or component.
- Persist only intentional, versioned, non-sensitive state.
- Selectors should expose domain meaning rather than store shape.

9. Authentication and Authorization

Design sessions and permissions around server trust boundaries.

Session architecture

- Use server-issued Secure, HttpOnly, SameSite cookies.
- Validate authentication and authorization on every protected server operation.
- Middleware may reject obvious unauthenticated navigation, but backend authorization remains final.
- Centralize 401/403 behaviour and clear only safe client caches.

Authorization model

Model permissions as capabilities rather than scattered role string checks.

UI visibility improves experience but never replaces server enforcement.

Sensitive operations require explicit audit events.

Threat controls

- CSRF protection for cookie-authenticated mutations, CSP, output encoding, HTML sanitization, clickjacking protection, safe redirects, and secure logout.

10. Frontend Security

Apply practical controls against browser, dependency, and content threats.

Security checklist

- No secrets in NEXT_PUBLIC variables or client bundles.
- No security claims based on client-side encryption with bundled keys.
- Sanitize user-generated HTML using allow-list policies.
- Validate upload type, extension, MIME, size, dimensions, and malware status.
- Pin and govern third-party scripts.
- Mask tokens, personal data, and message content from logs.

Supply-chain security

Use locked dependencies, automated vulnerability scanning, secret scanning, provenance-aware packages, and deliberate upgrade windows.

Remove duplicate libraries to reduce attack surface and maintenance cost.

11. Performance Engineering

Establish measurable budgets and optimise the routes that matter most.

Core practices

- Publish route bundle reports for production builds.
- Dynamically load editors, builders, charts, media tools, and large dialogs.
- Virtualize large tables, logs, dropdowns, and node lists.
- Use next/image and next/font correctly.
- Debounce searches, cancel stale requests, and deduplicate stable reference data.

Suggested budgets

Metric	Initial gate	Target
LCP p75	< 3.0 s	< 2.5 s
INP p75	< 300 ms	< 200 ms
CLS p75	< 0.15	< 0.10
Largest route JS	Baseline - 15%	No unapproved regression
Duplicate requests	Measured	Zero per interaction

Performance review questions

- What JavaScript is required before the user can act?
- Which data can be fetched on the server?
- Which dependency dominates the bundle?
- Does the component render thousands of DOM nodes?
- Can the route stream useful content earlier?

12. Testing Strategy

Protect critical behaviours at the most cost-effective test layer.

Test pyramid

Layer	Primary tools	Best targets
Unit	Vitest/Jest	Pure business logic, reducers, mapping
Component	React Testing Library	Forms, tables, dialogs, states
Integration	MSW + app tests	API client, auth expiry, caching
End-to-end	Playwright	Login, create, publish, payment-critical flows
Accessibility	axe + keyboard checks	Navigation, modals, forms, tables
Visual	Storybook/Playwright	Shared UI and critical screens

Definition of done

- Every bug fix includes a regression test when feasible.
- Every new business rule has unit or integration coverage.
- Critical journeys run in CI against stable fixtures.
- Flaky tests are defects, not acceptable noise.

13. Accessibility

Make keyboard, screen reader, contrast, and motion support part of normal engineering.

Standards

- Target WCAG 2.2 AA.
- Use semantic HTML before ARIA.
- Trap and restore focus in modal experiences.
- Associate labels, help text, required state, and errors correctly.
- Expose table headers, sort state, row actions, and responsive alternatives.
- Provide text or tabular alternatives for charts.
- Respect reduced motion and never communicate status by colour alone.

Accessibility review flow

- Keyboard-only pass
- Screen-reader name and role check
- Automated axe scan
- Contrast verification
- Zoom and reflow check
- Error-state review

14. Forms and Validation

Build predictable forms that scale from simple settings to multi-step workflows.

Form architecture

- Choose one primary form library.
- Define runtime schemas shared with server contracts where practical.
- Keep field components responsible for labels, help text, errors, and accessibility relationships.
- Separate draft state from submitted server state.

Validation rules

- Validate on the server regardless of client validation.
- Show field-level errors near the field and a summary for large forms.
- Do not block typing with expensive synchronous validation.
- Preserve user input after recoverable server errors.

15. Data Tables and Search Experiences

Design dense enterprise interfaces without sacrificing usability or performance.

Table standards

- Sticky headers, deliberate column widths, tooltips for truncation, accessible sort state, and full-width responsive behaviour.
- Use server pagination for large datasets and virtualization when rendering large visible sets.
- Keep actions predictable and confirm destructive operations.

Search standards

Debounce user-driven requests, cancel stale queries, sync filters to the URL, preserve back-navigation state, and define empty versus no-result states.

16. Observability and Operational Reliability

Make frontend failures traceable to releases, requests, routes, and user impact.

Required signals

- Centralized error tracking with source maps and release versions.
- Core Web Vitals by browser, geography, route, and account segment.
- Correlation IDs propagated across frontend and backend.
- Structured audit events for sensitive actions.
- Feature flags with owners, expiry dates, and kill switches.

Operational dashboards

JavaScript error rate, API error rate, slow endpoints, failed deployments, route performance, and critical flow conversion.

17. CI/CD and Release Engineering

Convert standards into automatic gates and safe deployment workflows.

Pipeline

1. Install locked dependencies
2. Validate environment schema
3. Lint with zero warnings
4. Typecheck
5. Unit and component tests
6. Production build
7. Bundle-budget check
8. Accessibility smoke tests
9. Critical E2E tests
10. Dependency and secret scans
11. Publish build metadata
12. Deploy with rollback support

Release controls

- Preview environments for pull requests.
- Progressive rollouts for risky features.
- Documented smoke tests and rollback ownership.
- Release notes tied to observable behaviour and migrations.

18. Developer Experience and Repository Governance

Make the correct workflow discoverable and fast.

Repository essentials

- README, architecture decisions, feature READMEs, environment catalogue, UI standards, API standards, testing handbook, and release runbook.
- CODEOWNERS for auth, shared UI, API foundations, and major domains.
- Consistent scripts: dev, build, lint, lint:fix, typecheck, test, test:e2e, analyze, validate.

Pull request expectations

- Small reviewable scope, linked requirement, screenshots for UI changes, test evidence, accessibility notes, performance impact, migration or rollback plan.

19. Code Review Standard

Review for behaviour, maintainability, security, and operational impact.

Review checklist

- Does the implementation respect module boundaries?
- Are permissions enforced server-side?
- Are external inputs validated?
- Are loading, empty, error, and permission states handled?
- Is the bundle impact justified?
- Can the behaviour be tested at a cheaper layer?
- Are accessibility and keyboard interactions correct?
- Is observability sufficient for production support?

20. Legacy Modernization

Improve a large codebase while continuing product delivery.

Migration sequence

- Baseline current risks and metrics.
- Stop new debt with CI gates.
- Build shared foundations: session, API, errors, tokens, testing.
- Migrate one business feature at a time.
- Remove adapters and duplicate libraries after consumers move.
- Track deprecations with owners and dates.

Risk controls

Risk	Control
Refactor delays delivery	Capacity allocation and feature-by-feature migration
Strict TypeScript creates noise	Baseline and touched-file policy
Design system becomes cosmetic	Connect tokens to primitives, tests, and accessibility
Session migration disrupts users	Staged rollout, telemetry, rollback
Tests become flaky	Stable data, ownership, deadlines
Legacy patterns keep growing	Import restrictions, templates, CODEOWNERS

21. Reference Implementation Playbook

A practical sequence for establishing the architecture in a real repository.

First 20 engineering tickets

- Inventory browser storage and session usage.
- Classify environment variables.
- Design server-backed session contract.
- Centralize unauthorized-session handling.
- Add CSP and security headers.
- Make lint and typecheck mandatory.
- Create a no-growth warning baseline.
- Add dependency and secret scans.
- Create design-token files.
- Move fonts to next/font.
- Create shared form primitives.
- Bridge tokens into the UI theme.
- Create normalized API errors.
- Consolidate HTTP clients.
- Introduce critical endpoint schemas.
- Adopt server-state caching.
- Publish feature-module template.
- Add bundle analysis.
- Lazy-load heavy dependencies.
- Add authentication and critical-flow tests.

Professional definition of done

- Acceptance criteria and edge cases are documented.
- Protected operations enforce authorization server-side.
- No sensitive browser storage or public secrets are introduced.
- No new unsafe typing or hardcoded design values without justification.
- Loading, empty, success, error, permission, and timeout states are handled.
- Keyboard, focus, labels, and contrast are verified.
- Tests and monitoring are updated.
- CI passes with no quality-gate violations.

22. Portfolio Case Study Guidance

Present architectural work credibly and without exposing proprietary information.

Positioning

- Describe the handbook as a vendor-neutral engineering blueprint informed by practical experience.
- Focus on decisions, trade-offs, principles, and reusable examples rather than claiming ownership of confidential systems.
- Link the portfolio case study to the GitHub chapters and downloadable PDF.

Public disclaimer

This handbook is an independent engineering reference created for educational and portfolio purposes. It does not disclose proprietary source code, confidential company information, customer data, credentials, or organisation-specific implementation details.

Appendix A. Architecture Review Checklist

Reusable reference for project delivery, reviews and engineering governance.

- Business capability and owner are clear
- Module boundary is explicit
- Rendering strategy is documented
- Data ownership is defined
- Failure modes are handled
- Security boundary is server-enforced
- Performance impact is measured
- Migration and rollback are documented

How to use this template

Copy this checklist into an issue, pull request, architecture review, or team runbook. Add an owner and evidence for every completed item. Any intentionally skipped item should include a short risk-based explanation.

Evidence examples

Links to tests, screenshots, performance reports, monitoring dashboards, architecture decisions, security reviews, release notes, or rollback instructions.

Appendix B. Pull Request Template

Reusable reference for project delivery, reviews and engineering governance.

- Linked requirement and acceptance criteria
- Scope and architectural impact
- Screenshots or recordings for UI changes
- Testing evidence
- Accessibility verification
- Security and privacy impact
- Bundle or performance impact
- Deployment and rollback notes

How to use this template

Copy this checklist into an issue, pull request, architecture review, or team runbook. Add an owner and evidence for every completed item. Any intentionally skipped item should include a short risk-based explanation.

Evidence examples

Links to tests, screenshots, performance reports, monitoring dashboards, architecture decisions, security reviews, release notes, or rollback instructions.

Appendix C. Feature README Template

Reusable reference for project delivery, reviews and engineering governance.

- Purpose and business capability
- Routes and entry points
- Public exports and boundaries
- API contracts and repositories
- State ownership
- Permissions
- Analytics and observability
- Testing strategy
- Known limitations and migration notes

How to use this template

Copy this checklist into an issue, pull request, architecture review, or team runbook. Add an owner and evidence for every completed item. Any intentionally skipped item should include a short risk-based explanation.

Evidence examples

Links to tests, screenshots, performance reports, monitoring dashboards, architecture decisions, security reviews, release notes, or rollback instructions.

Appendix D. Architecture Decision Record Template

Reusable reference for project delivery, reviews and engineering governance.

- Status and date
- Context and problem statement
- Decision drivers
- Considered options
- Selected decision
- Positive and negative consequences
- Migration plan
- Owner and review date

How to use this template

Copy this checklist into an issue, pull request, architecture review, or team runbook. Add an owner and evidence for every completed item. Any intentionally skipped item should include a short risk-based explanation.

Evidence examples

Links to tests, screenshots, performance reports, monitoring dashboards, architecture decisions, security reviews, release notes, or rollback instructions.

Appendix E. API Endpoint Review Checklist

Reusable reference for project delivery, reviews and engineering governance.

- Typed request and response
- Runtime validation at trust boundary
- Normalized errors
- Cancellation and timeout
- Retry policy
- Idempotency for critical mutation
- Cache and invalidation plan
- Correlation ID and monitoring

How to use this template

Copy this checklist into an issue, pull request, architecture review, or team runbook. Add an owner and evidence for every completed item. Any intentionally skipped item should include a short risk-based explanation.

Evidence examples

Links to tests, screenshots, performance reports, monitoring dashboards, architecture decisions, security reviews, release notes, or rollback instructions.

Appendix F. Frontend Security Review Checklist

Reusable reference for project delivery, reviews and engineering governance.

- No secrets in client bundle
- Authorization enforced server-side
- External input validated
- HTML sanitized
- Uploads validated
- Safe redirects
- CSP compatibility
- Sensitive data masked from logs

How to use this template

Copy this checklist into an issue, pull request, architecture review, or team runbook. Add an owner and evidence for every completed item. Any intentionally skipped item should include a short risk-based explanation.

Evidence examples

Links to tests, screenshots, performance reports, monitoring dashboards, architecture decisions, security reviews, release notes, or rollback instructions.

Appendix G. Accessibility Acceptance Checklist

Reusable reference for project delivery, reviews and engineering governance.

- Keyboard-only completion
- Visible focus
- Correct accessible names
- Labels and errors associated
- Modal focus trapped and restored
- Contrast meets target
- Zoom and reflow supported
- Reduced motion respected

How to use this template

Copy this checklist into an issue, pull request, architecture review, or team runbook. Add an owner and evidence for every completed item. Any intentionally skipped item should include a short risk-based explanation.

Evidence examples

Links to tests, screenshots, performance reports, monitoring dashboards, architecture decisions, security reviews, release notes, or rollback instructions.

Appendix H. Performance Review Checklist

Reusable reference for project delivery, reviews and engineering governance.

- Route JavaScript measured
- Heavy dependencies lazy-loaded
- Server rendering considered
- Images and fonts optimized
- Long lists virtualized
- Duplicate requests removed
- Searches debounced and cancelled
- Web Vitals observed

How to use this template

Copy this checklist into an issue, pull request, architecture review, or team runbook. Add an owner and evidence for every completed item. Any intentionally skipped item should include a short risk-based explanation.

Evidence examples

Links to tests, screenshots, performance reports, monitoring dashboards, architecture decisions, security reviews, release notes, or rollback instructions.

Appendix I. Testing Plan Template

Reusable reference for project delivery, reviews and engineering governance.

- Business risks ranked
- Unit targets identified
- Component states covered
- Integration boundaries mocked responsibly
- Critical E2E journeys defined
- Accessibility checks included
- Fixtures and test data owned
- Flakiness response documented

How to use this template

Copy this checklist into an issue, pull request, architecture review, or team runbook. Add an owner and evidence for every completed item. Any intentionally skipped item should include a short risk-based explanation.

Evidence examples

Links to tests, screenshots, performance reports, monitoring dashboards, architecture decisions, security reviews, release notes, or rollback instructions.

Appendix J. Production Readiness Checklist

Reusable reference for project delivery, reviews and engineering governance.

- Environment validation passes
- Monitoring and source maps enabled
- Feature flags configured
- Smoke tests documented
- Rollback verified
- Security scans clean
- Performance budgets pass
- Support and incident ownership assigned

How to use this template

Copy this checklist into an issue, pull request, architecture review, or team runbook. Add an owner and evidence for every completed item. Any intentionally skipped item should include a short risk-based explanation.

Evidence examples

Links to tests, screenshots, performance reports, monitoring dashboards, architecture decisions, security reviews, release notes, or rollback instructions.

Appendix K. Incident Response Guide

Reusable reference for project delivery, reviews and engineering governance.

- Confirm impact and affected routes
- Identify release and correlation IDs
- Disable risky feature through kill switch
- Rollback when recovery is faster than diagnosis
- Preserve logs and evidence
- Communicate status
- Create root-cause analysis
- Add regression protection

How to use this template

Copy this checklist into an issue, pull request, architecture review, or team runbook. Add an owner and evidence for every completed item. Any intentionally skipped item should include a short risk-based explanation.

Evidence examples

Links to tests, screenshots, performance reports, monitoring dashboards, architecture decisions, security reviews, release notes, or rollback instructions.

Appendix L. Dependency Adoption Checklist

Reusable reference for project delivery, reviews and engineering governance.

- Clear business or engineering need
- Bundle and runtime cost measured
- Maintenance activity reviewed
- Security posture reviewed
- Licence compatible
- SSR and browser compatibility verified
- Alternative native capability considered
- Removal strategy documented

How to use this template

Copy this checklist into an issue, pull request, architecture review, or team runbook. Add an owner and evidence for every completed item. Any intentionally skipped item should include a short risk-based explanation.

Evidence examples

Links to tests, screenshots, performance reports, monitoring dashboards, architecture decisions, security reviews, release notes, or rollback instructions.

Appendix M. Design Token Proposal Template

Reusable reference for project delivery, reviews and engineering governance.

- Problem and repeated visual decision
- Semantic token name
- Light and dark values
- Accessibility impact
- Affected components
- Migration plan
- Design approval
- Documentation and visual test update

How to use this template

Copy this checklist into an issue, pull request, architecture review, or team runbook. Add an owner and evidence for every completed item. Any intentionally skipped item should include a short risk-based explanation.

Evidence examples

Links to tests, screenshots, performance reports, monitoring dashboards, architecture decisions, security reviews, release notes, or rollback instructions.

Appendix N. Shared Component Readiness Checklist

Reusable reference for project delivery, reviews and engineering governance.

- Clear responsibility
- Typed API
- All visual and interaction states
- Keyboard and screen reader support
- Responsive behaviour
- Theme support
- Unit or interaction tests
- Storybook documentation

How to use this template

Copy this checklist into an issue, pull request, architecture review, or team runbook. Add an owner and evidence for every completed item. Any intentionally skipped item should include a short risk-based explanation.

Evidence examples

Links to tests, screenshots, performance reports, monitoring dashboards, architecture decisions, security reviews, release notes, or rollback instructions.

Appendix O. Table Component Checklist

Reusable reference for project delivery, reviews and engineering governance.

- Semantic headers and caption
- Accessible sort state
- Deliberate widths
- Sticky header behaviour
- Truncation tooltips
- Pagination or virtualization
- Responsive alternative
- Loading, empty and error states

How to use this template

Copy this checklist into an issue, pull request, architecture review, or team runbook. Add an owner and evidence for every completed item. Any intentionally skipped item should include a short risk-based explanation.

Evidence examples

Links to tests, screenshots, performance reports, monitoring dashboards, architecture decisions, security reviews, release notes, or rollback instructions.

Appendix P. Form Component Checklist

Reusable reference for project delivery, reviews and engineering governance.

- Labels and descriptions
- Required and disabled states
- Field and summary errors
- Server validation
- Input preservation after failure
- Submission loading state
- Keyboard order
- Analytics only where justified

How to use this template

Copy this checklist into an issue, pull request, architecture review, or team runbook. Add an owner and evidence for every completed item. Any intentionally skipped item should include a short risk-based explanation.

Evidence examples

Links to tests, screenshots, performance reports, monitoring dashboards, architecture decisions, security reviews, release notes, or rollback instructions.

Appendix Q. Authentication Migration Plan

Reusable reference for project delivery, reviews and engineering governance.

- Inventory existing token storage
- Define server session contract
- Introduce cookie-based session
- Add centralized unauthorized handling
- Migrate protected routes
- Migrate permissions
- Remove legacy storage
- Monitor and roll back safely

How to use this template

Copy this checklist into an issue, pull request, architecture review, or team runbook. Add an owner and evidence for every completed item. Any intentionally skipped item should include a short risk-based explanation.

Evidence examples

Links to tests, screenshots, performance reports, monitoring dashboards, architecture decisions, security reviews, release notes, or rollback instructions.

Appendix R. Strict TypeScript Migration Plan

Reusable reference for project delivery, reviews and engineering governance.

- Capture unsafe baseline
- Enable strict rules in new modules
- Validate API boundaries
- Replace broad any types
- Use expect-error with expiry only when necessary
- Add no-growth CI rule
- Migrate high-change modules first
- Remove temporary compatibility types

How to use this template

Copy this checklist into an issue, pull request, architecture review, or team runbook. Add an owner and evidence for every completed item. Any intentionally skipped item should include a short risk-based explanation.

Evidence examples

Links to tests, screenshots, performance reports, monitoring dashboards, architecture decisions, security reviews, release notes, or rollback instructions.

Appendix S. Design System Rollout Plan

Reusable reference for project delivery, reviews and engineering governance.

- Inventory repeated UI patterns
- Establish token foundation
- Create core primitives
- Bridge third-party theme
- Document states in Storybook
- Migrate high-traffic screens
- Add visual regression tests
- Remove duplicate component libraries

How to use this template

Copy this checklist into an issue, pull request, architecture review, or team runbook. Add an owner and evidence for every completed item. Any intentionally skipped item should include a short risk-based explanation.

Evidence examples

Links to tests, screenshots, performance reports, monitoring dashboards, architecture decisions, security reviews, release notes, or rollback instructions.

Appendix T. Performance Optimisation Plan

Reusable reference for project delivery, reviews and engineering governance.

- Capture route baseline
- Rank largest bundles
- Remove artificial delays
- Lazy-load heavy tools
- Move data work server-side
- Optimise images and fonts
- Virtualize dense interfaces
- Enforce budgets in CI

How to use this template

Copy this checklist into an issue, pull request, architecture review, or team runbook. Add an owner and evidence for every completed item. Any intentionally skipped item should include a short risk-based explanation.

Evidence examples

Links to tests, screenshots, performance reports, monitoring dashboards, architecture decisions, security reviews, release notes, or rollback instructions.

Appendix U. Testing Adoption Plan

Reusable reference for project delivery, reviews and engineering governance.

- Start with auth and one critical journey
- Add unit coverage for business rules
- Build reusable MSW handlers
- Create stable fixtures
- Run critical Playwright flows in CI
- Add accessibility smoke checks
- Track flakes as defects
- Expand changed-code coverage

How to use this template

Copy this checklist into an issue, pull request, architecture review, or team runbook. Add an owner and evidence for every completed item. Any intentionally skipped item should include a short risk-based explanation.

Evidence examples

Links to tests, screenshots, performance reports, monitoring dashboards, architecture decisions, security reviews, release notes, or rollback instructions.

Appendix V. Release Runbook Template

Reusable reference for project delivery, reviews and engineering governance.

- Release scope
- Database or contract dependencies
- Feature flags
- Pre-deployment checks
- Deployment steps
- Smoke tests
- Monitoring window
- Rollback trigger and owner

How to use this template

Copy this checklist into an issue, pull request, architecture review, or team runbook. Add an owner and evidence for every completed item. Any intentionally skipped item should include a short risk-based explanation.

Evidence examples

Links to tests, screenshots, performance reports, monitoring dashboards, architecture decisions, security reviews, release notes, or rollback instructions.

Appendix W. Environment Variable Catalogue

Reusable reference for project delivery, reviews and engineering governance.

- Variable name
- Purpose
- Public or private classification
- Required environments
- Default allowed
- Owner
- Rotation or update process
- Validation rule

How to use this template

Copy this checklist into an issue, pull request, architecture review, or team runbook. Add an owner and evidence for every completed item. Any intentionally skipped item should include a short risk-based explanation.

Evidence examples

Links to tests, screenshots, performance reports, monitoring dashboards, architecture decisions, security reviews, release notes, or rollback instructions.

Appendix X. Observability Event Standard

Reusable reference for project delivery, reviews and engineering governance.

- Event name and business meaning
- Trigger condition
- Required properties
- Sensitive fields excluded
- Sampling policy
- Owner and dashboard
- Retention expectation
- Versioning and deprecation

How to use this template

Copy this checklist into an issue, pull request, architecture review, or team runbook. Add an owner and evidence for every completed item. Any intentionally skipped item should include a short risk-based explanation.

Evidence examples

Links to tests, screenshots, performance reports, monitoring dashboards, architecture decisions, security reviews, release notes, or rollback instructions.

Appendix Y. Engineering Maturity Review

Reusable reference for project delivery, reviews and engineering governance.

- Security boundaries
- Architecture consistency
- Type safety
- Testing depth
- Performance discipline
- Accessibility coverage
- Release reliability
- Documentation and ownership

How to use this template

Copy this checklist into an issue, pull request, architecture review, or team runbook. Add an owner and evidence for every completed item. Any intentionally skipped item should include a short risk-based explanation.

Evidence examples

Links to tests, screenshots, performance reports, monitoring dashboards, architecture decisions, security reviews, release notes, or rollback instructions.